chapter V

Git & Github

No More ‘Final_Final_v3.zip

Full Name

Mechkak Hocine Arezki

Email

hocine182003@gmail.com

Website

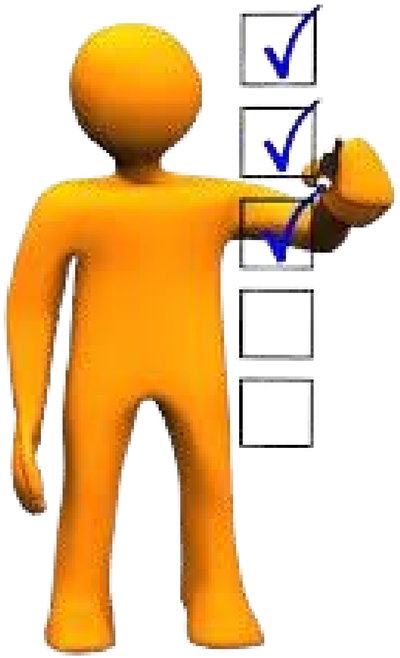
https://mechkakhocine.vercel.app/

Essential Developer Tools

1. **code editors**
   - VS Code : Most popular choice, free & extensible
   - IntelliJ IDEA : Best for Java development
   - Sublime Text : Freemium, lightweight, similar to VS Code
   - vim/neovim : Powerful, keyboard-driven editors for hardcore devs
2. **Terminals & Shells**
   - Command Prompt / PowerShell (Windows)
   - Terminal (Linux/macOS)
3. **Debugging**
   - Browser DevTools : Built into Chrome, Brave, Firefox, etc. for frontend debugging
4. **Testing Tools**
   - Insomnia / Postman : API testing tools
5. **AI Tools for Assistance**
   - ChatGPT / Claude AI : Help with coding questions, debugging, and explanations
6. **Where to Get Help**
   - Stack Overflow : Ask & find coding solutions
   - Quora : General discussions & programming insights
   - Official Documentation : Best source for frameworks/libraries
   - GitHub : Explore open-source projects & issues

Prerequisites for Git & Github

- you need to have an account of github
- you need to download the Git tool
- you need to have vscode IDE



What is Github ?

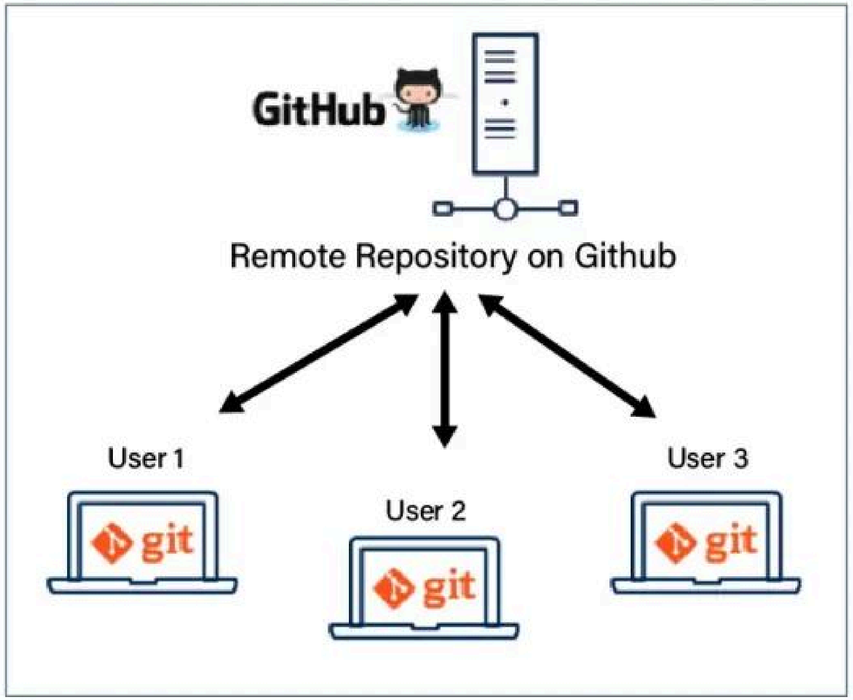
Github a cloud-based platform (website) built on the Git version control system that allows developers and other users to store, manage, share, and collaborate on code and projects. It acts as a central hub for software development, offering tools for version control, project management, and team collaboration.

What is Git ?

Git is a distributed version control system (DVCS) designed to help developers track changes, manage code history, and work together efficiently without messing up each other’s work ensuring that their changes don't conflict with each other.

How Git & Github works together

Git tracks and manages changes on your local computer, keeping a record of your work as you develop. When you push those changes, GitHub stores them online in a shared repository, making them accessible to others and enabling collaboration across teams or contributors.



What should i use Git & Github ?

- Collaboration – Work with teams easily without overwriting code
- Backup & Security – Store code safely in the cloud
- Branching & Experimenting – Create separate branches for new features without affecting the main code
- Open Source Contributions – Contribute to projects and showcase your work

Life before Git & Github

Before Git & GitHub

- Manually saving different versions of files (finalversion.zip)
- Sharing code through emails or USB drives
- Hard to collaborate without overwriting each other's work
- Difficult to track who made which changes

After Git & GitHub

- Git automatically tracks changes (no need for multiple copies)
- GitHub allows easy online collaboration
- Developers can work on different branches without conflicts
- Full history of code changes with commit messages

Setting Up Git

Before working with Git, we need to configure it on our system.

1. open git bash installed in your computer
2. type this two commands:

- git config --global user.email "your.email@example.com"
- git config --global user.name "Your Name"

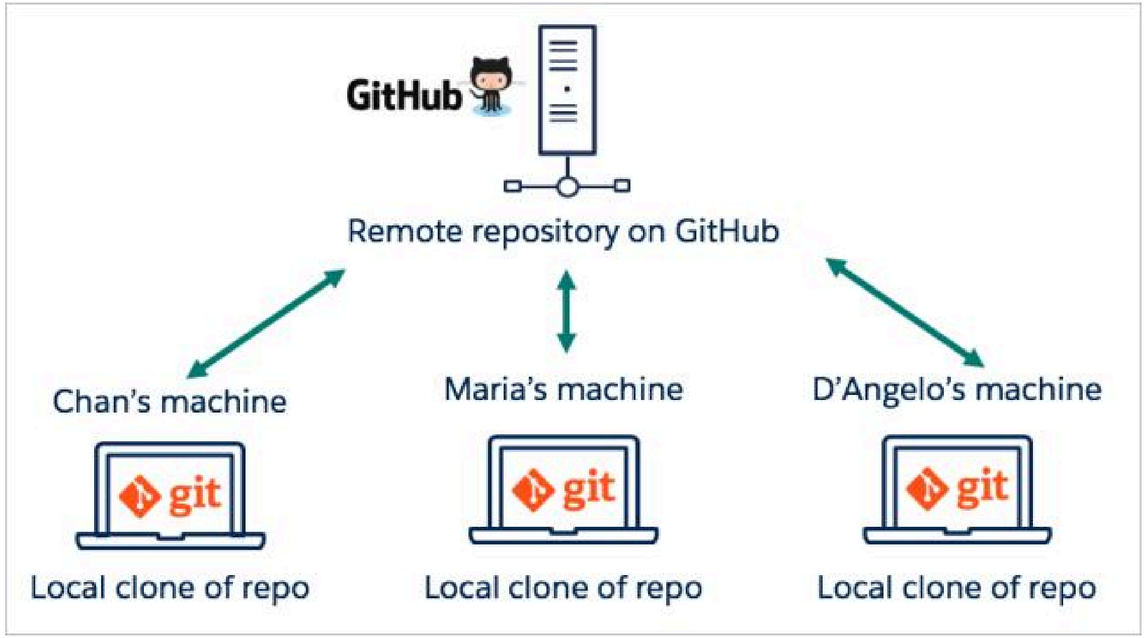
```
1  git config --global user.name "Your Name here"
2  git config --globl user.email "Your Email here"
```

importnant notes:

- The email must match the one used for your GitHub account.
- The name should be your real Github name, not your GitHub username.

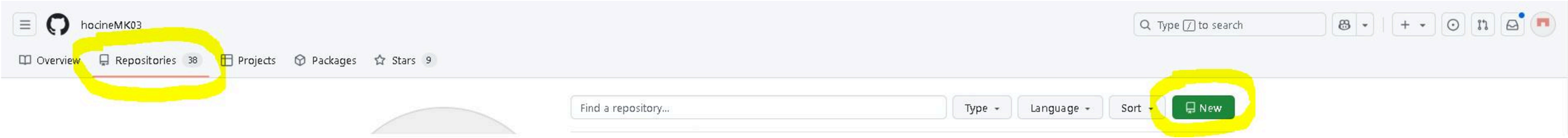
What is a Repository ?

A github repository is a place where you store your github project, It can be local (on your machine) or remote (stored in github)



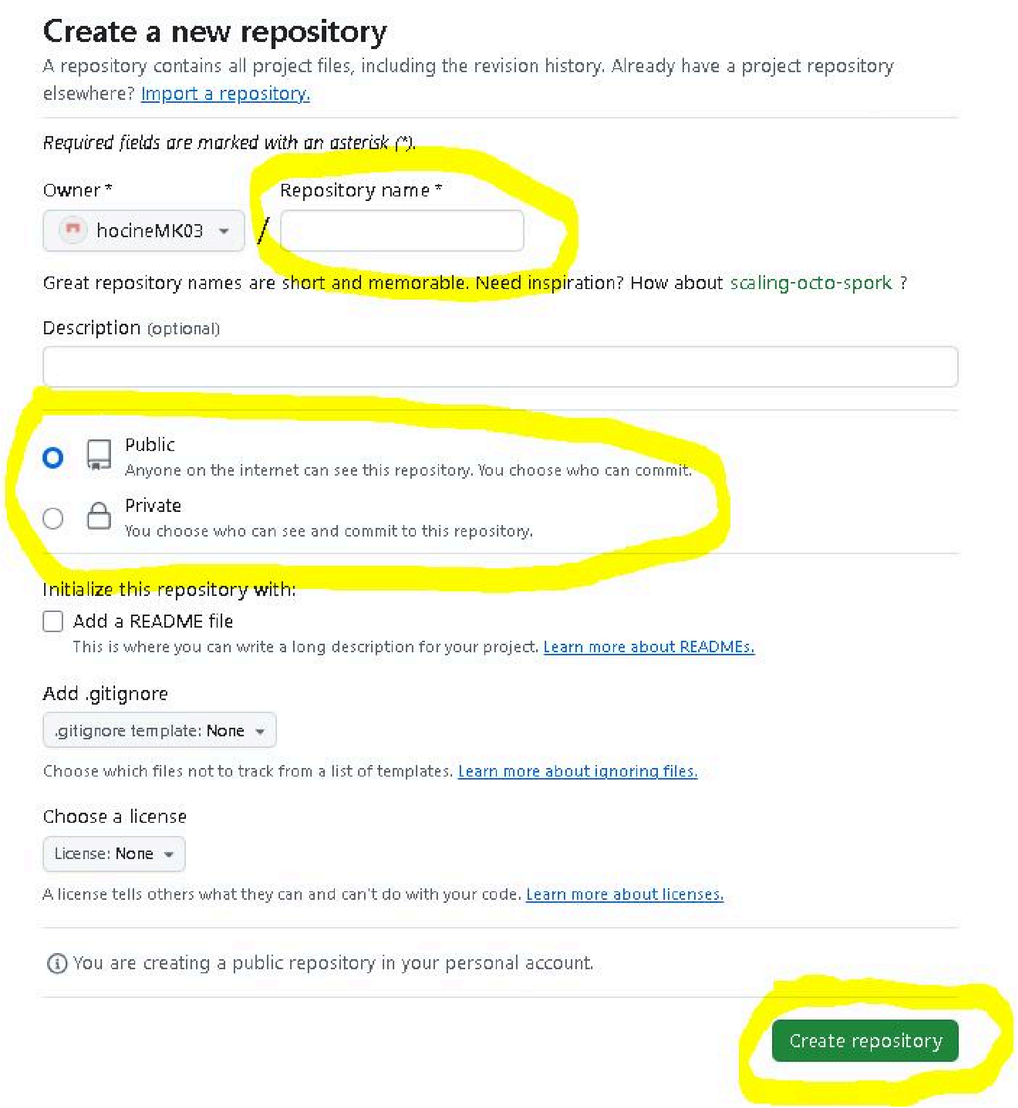
Creating a Repository

1. You need to login to your github account.
2. go to your profil and select repository.
3. click on the “ new “ button.



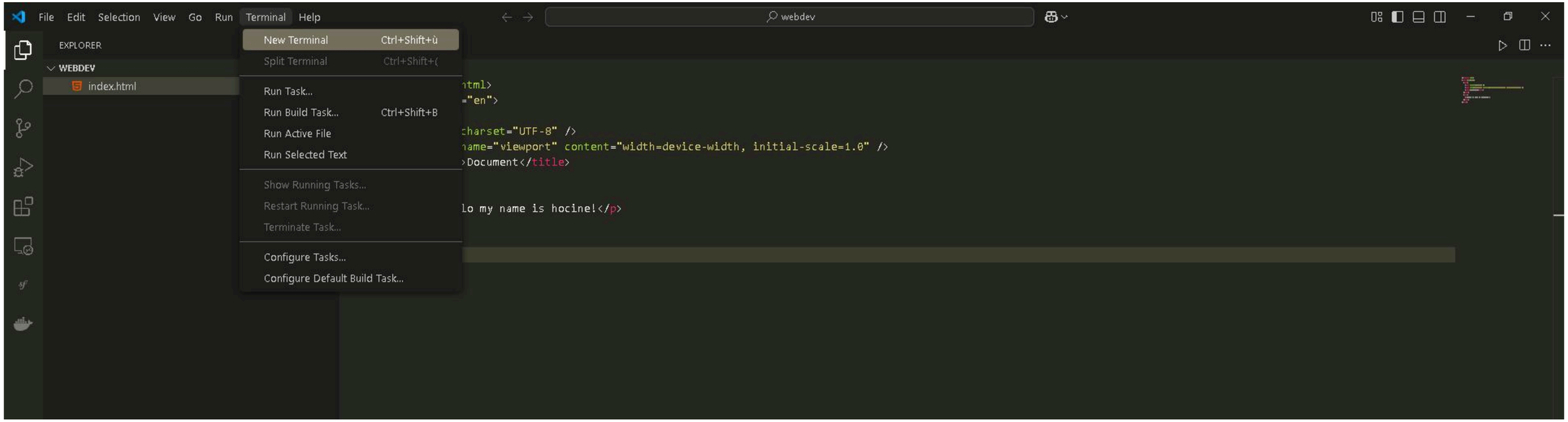
Creating a Repository

4. Give your new repository a name
5. select your repository visibility :
   - public : Anyone can see and access it.
   - private : Only you and your collaborators can access it.
6. click the create repository button to finish



Creating a Repository

7. go to your project folder
8. open it in vscode
9. open a new terminal



Creating a Repository

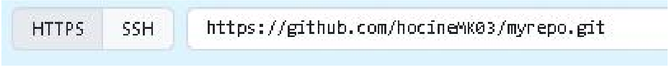
```
PS C:\Users\hocine\Desktop\webdev> git init
Initialized empty Git repository in C:/Users/hocine/Desktop/webdev/.git/
PS C:\Users\hocine\Desktop\webdev> git add .
PS C:\Users\hocine\Desktop\webdev> git commit -m " my first commit"
[master (root-commit) 48ee871]  my first commit
 1 file changed, 11 insertions(+)
 create mode 100644 index.html
PS C:\Users\hocine\Desktop\webdev>
```

10. the last step is to enter git commands :
    - git init : Initializes a Git repository in your folder, making it a Git project. This is the first and most important step.
    - git add . : Stages all files, telling Git they are ready to be committed.
    - git commit -m “message” : Saves the changes with a message. This helps in tracking commit history.

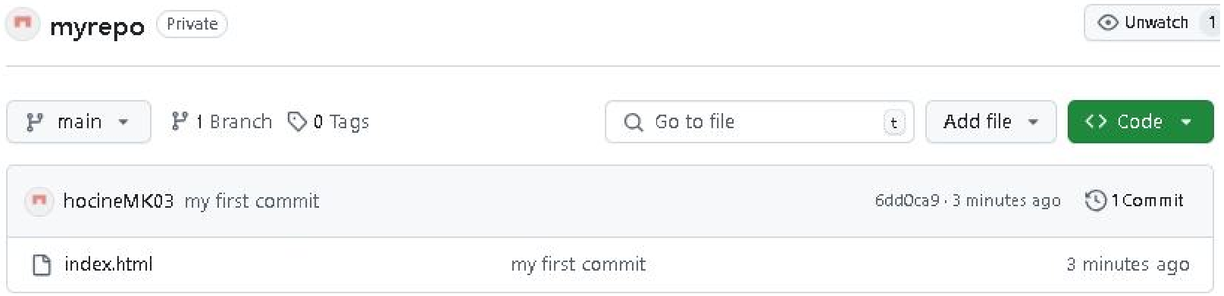
Creating a Repository

```
PS C:\Users\hocine\Desktop\webdev> git branch -m main
PS C:\Users\hocine\Desktop\webdev> git remote add origin https://github.com/hocineMK03/myrepo.git
PS C:\Users\hocine\Desktop\webdev> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 388 bytes | 388.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hocineMK03/myrepo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\hocine\Desktop\webdev>
```

- git branch -M main : Renames the current branch to main (or another name for consistency). This helps align with GitHub's default branch naming.
- git remote add origin : Links your local repository to the remote repository on GitHub.



- git push -u origin main : Uploads your project from your local repository to GitHub.



Updating a Repository

to update you follow this steps :

1. make your changes in your code
2. use git add . command
3. use git commit -m “message” command
4. use git push origin main command

notice how we used
git push origin main instead of
git push -u origin main, it is because -u flag is only required when it’s your first time you push and after that it’s optional.

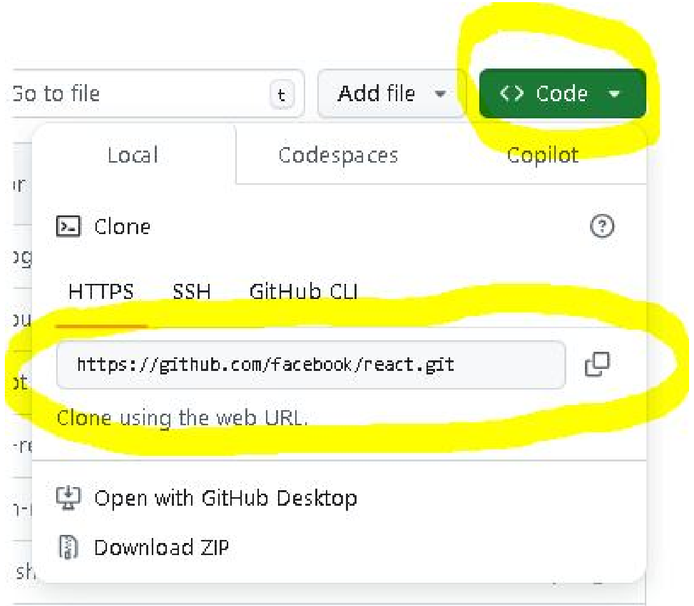
```
PS C:\Users\hocine\Desktop\webdev> git add .
PS C:\Users\hocine\Desktop\webdev> git commit -m " added a funcitonality"
[main afa1710]  added a funcitonality
 1 file changed, 1 insertion(+)
PS C:\Users\hocine\Desktop\webdev> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 556 bytes | 556.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/hocineMK03/myrepo.git
   c8ff06d..afa1710  main -> main
PS C:\Users\hocine\Desktop\webdev>
```

Cloning a Repository

Cloning allows you to copy a remote repository (from GitHub) to your local machine so you can work on it or to test it.

Steps for cloning are :

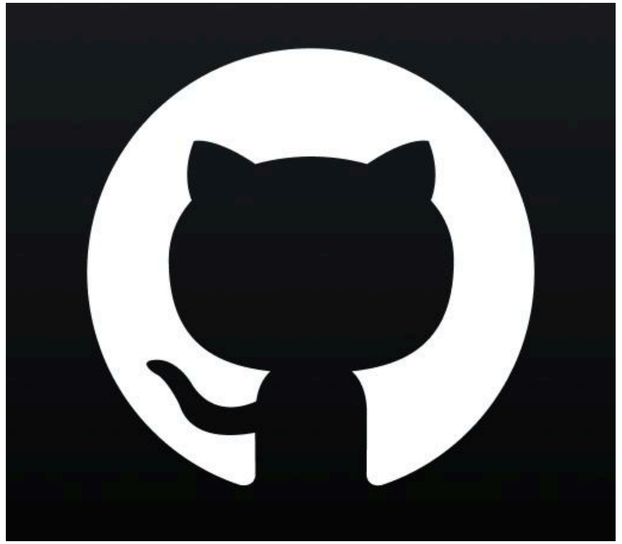
1. go to the repository you wan tot clone
2. click the <> Code button and copy the url
3. run the command git clone <repo url>



Contributing to Open Source

Why contribute ?

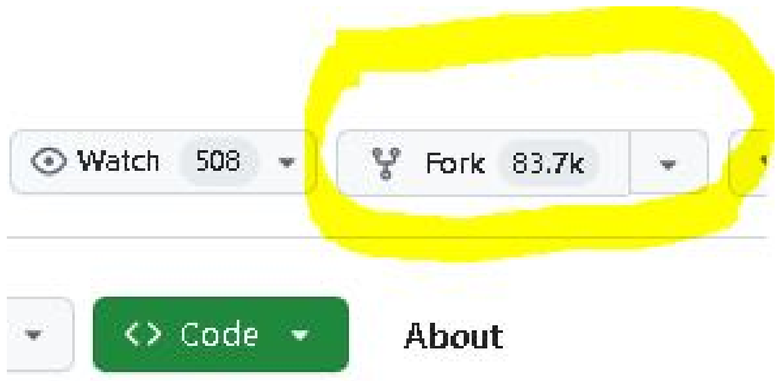
- Gain real-world coding experience.
- Collaborate with developers worldwide.
- Build a strong portfolio for jobs & internships.
- Improve your Git & GitHub skills.



Contributing to Open Source

We will learn how to contribute by using the popular repo for first time contributions : first contributions

1. go to the repository by clicking the link
2. press fork button to copy the repo to your github account
3. clone the fork using git clone <the url>
4. in terminal type cd first-contributions



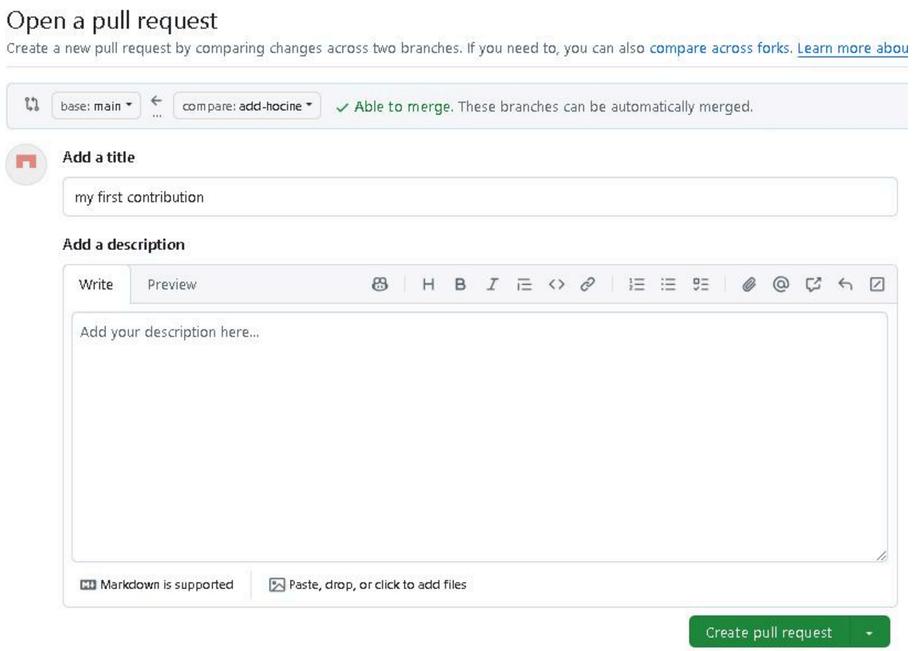
Contributing to Open Source

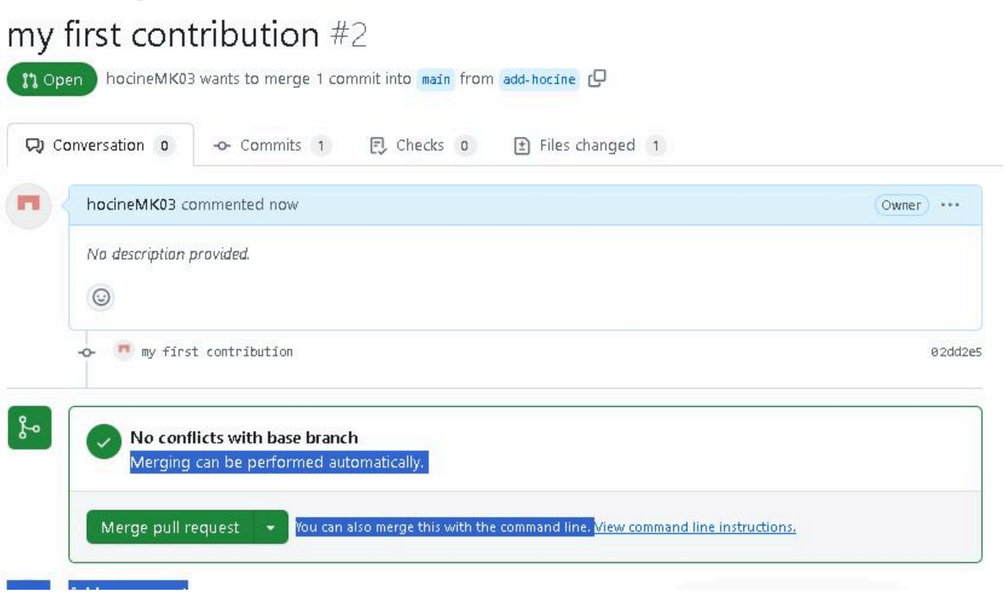
5. use git checkout -b add-your-name to create a new branch
6. make contribution to this repo
7. stage the files by using git add .
8. commit your files by using git commit -m “message”
9. push your changes by typing git push origin
10. press the compare & pull request button

```
PS C:\Users\hocine\Desktop\firstcontribution> git checkout -b add-hocine
Switched to a new branch 'add-hocine'
PS C:\Users\hocine\Desktop\firstcontribution> git add .
PS C:\Users\hocine\Desktop\firstcontribution> git commit -m "my first contribution"
[add-hocine 9f18efd] my first contribution
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\hocine\Desktop\firstcontribution> git push origin add-hocine
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (3/3), 299 bytes | 299.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'add-hocine' on GitHub by visiting:
remote:      https://github.com/hocineMK03/your-first-contribution/pull/new/add-hocine
remote:
To https://github.com/hocineMK03/your-first-contribution.git
 * [new branch]      add-hocine -> add-hocine
```

Contributing to Open Source

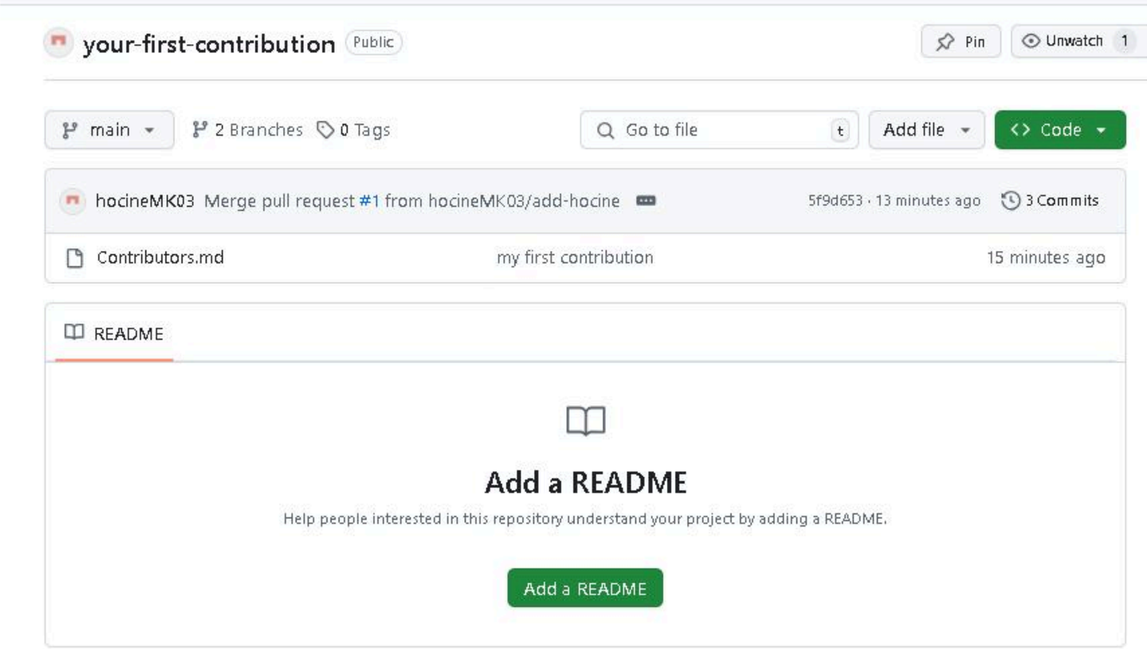
11. create a pull request
12. merge the pull request





Contributing to Open Source

now we try contributing using this repository, you can try here



End Of Chapter V